

Applications of Linear Programming Techniques to Satellite Power Management and Scheduling

By Joshua CRITCHLEY-MARROWS,¹⁾ Martin ISACSSON,¹⁾ and Agnes GÄRDEBÄCK¹⁾

¹⁾KTH Royal Technology Institute, Stockholm, Sweden

(Received June 23rd, 2017)

The power system is one of the most important subsystems for a successful space mission. Any failure in this subsystem leads to a direct loss of a satellite. This creates a need for a power schedule to be effectively and efficiently produced, especially if requirements are constantly changing. This paper presents the application of linear programming techniques to solving the power schedule problem, with the more specific usage of mixed-integer linear programming (MILP). The illustration of the approach is applied to a Swedish student satellite, which consists of the necessary subsystems and eight separate experiments. Two programs are developed, one studying the satellite lifetime in terms of orbital cycles and the other studying the individual orbit cycle. Simulating the lifetime of the satellite over 5000 orbit cycles, the battery level did not decline below 76.35%. Using a computer with an Intel i4 processor, this simulation took 3.2 hrs, with individual orbits taking 2.3 s each. Further work includes developing the program to be completed on-board the satellite, adapting to new scenarios, and incorporating a model for the decline of battery performance over time.

Key Words: Linear Programming, Satellite Power Management, Satellite Power Scheduling

Nomenclature

A	: matrix of linear constraint coefficients
B	: battery energy stored in an orbit
E	: specified component energy requirement
N	: desired number for a constraint
P	: specified component power requirement
R	: desired time for a constraint
T	: number of time increments
a	: arbitrary coefficient
e	: number of time increments in eclipse
f	: objective function
l	: arbitrary upper limit
n	: number of satellite components
s	: percentage requirement
x	: optimisable variable, component state
y	: optimisable variable, begin operation
ΔB	: difference in battery energy
Δt	: size of time increment
E	: array of component energy for each component state
LB	: array of variable lower bounds
LS	: matrix mapping processes to orbits
P	: array of component power requirements
S	: matrix of components on/off commands
UB	: array of variable upper bounds
b	: array of linear constraint equalities
x	: array of optimisable variables, array of component states

Subscripts

i	: component number
j	: time step number

1. Introduction

Effective power scheduling and management is vital to the success of any space mission. Organisation of this system is dependent on the requirements of each satellite subsystem and, more specifically, the needs of the space mission payload. Power is a limiting resource to the satellite whilst in orbit, and it is fundamental that the power management procedure applied to the satellite is both efficient and effective.

Throughout the design procedure, various components of the subsystems are modified and changed, requiring a new power schedule to be frequently updated and revised, especially since each subsystem is interdependent with others to at least some degree. Additionally, mission requirements can change even after the satellite has been launched, and a new power plan must be uploaded to the device. In such situations, it is of interest for this update to the power schedule to be fast and precisely produced, especially when the system requirements and dependencies are complex.

Previous work on this topic is not extensive. For the different case of satellite communication scheduling, Xhafa *et al.* proposed using genetic algorithms to this problem.¹⁾ Other optimisation tools have been developed for the cases of mission planning,²⁾ and energy budgeting,³⁾ using constraint based approaches or field programmable gate arrays respectively, but not for the specific problem of power scheduling.

This paper presents the application of linear programming techniques to solve the power scheduling problem, specifically using mixed-integer linear programming (MILP). This approach requires conversion of constraint problems to linear inequalities, with predominantly integer-based solutions. This technique allows for the use of simple algorithms to solve the problem, where matrix arrays are developed for power requirements.

The illustration of the approach is performed through application to the MIniatUre STudent (MIST) satellite being designed and built by the KTH (Kungliga Tekniska Högskolan) Royal Technology Institute in Sweden. With MIST consisting of eight separate payloads being confined within a three-unit CubeSat, as well as necessary subsystems (e.g. communications), the logistical challenge of developing a constantly changing power schedule was large, and required use of a technique such as the MILP approach.

For instance, the satellite camera must be used N times during the lifetime. However, it is not to be used at the same time as CUBEPROP, a prototype small satellite propulsion unit developed at Nanospace in Uppsala, Sweden. Such a requirement must be changed to the MILP format, so it may be processed by the optimisation algorithm.

This application will be illustrated by describing some of the various subsystem requirements of the MIST satellite. These subsystems are summarised in Table 1. The selection was made to highlight different features and applications within the constraints.

Table 1. Description of various subsystems used in the MIST satellite.

Component	Description
CUBES	Radiation monitoring
RATEX-J	Radiation detection experimental unit
SiC in Space	Semiconductor electronics experiment
LEGS	Piezoelectric motor experiment
MOREBAC	Measure micro-organisms growth characteristics in orbit
Camera	Capture high quality orbital pictures from MIST
SEUD	Single event upset detection
CUBEPROP	Prototype small satellite propulsion system

Each component will involve one or several derivations corresponding to each requirement, converting it to the necessary MILP form. Results of application of the MILP approach are included to emphasise the practicality of the technique. The details of this paper are concerned with illustrating the technique and its applications, not to give a procedure to development of a program.

2. Introduction to MILP

Linear programming generally requires the definition of an objective function which is either to be minimized or maximized. This function must be linear, with a variable $\mathbf{x} = (x_1, x_2, \dots, x_n)$ that will be optimised to satisfy an objective function. This objective function $f(\mathbf{x})$ will take the form,

$$f(\mathbf{x}) = a_1x_1 + a_2x_2 + \dots + a_nx_n. \quad (1)$$

This objective function is also constrained by a set of linear equalities of the form,

$$A\mathbf{x} \leq \mathbf{b}, \quad (2)$$

and,

$$LB \leq \mathbf{x} \leq UB. \quad (3)$$

By defining $f(\mathbf{x})$, A , $\mathbf{b}$, LB , and UB , an optimization algorithm may be used to find the best variables that satisfy

these constraints.

More specifically in this application, mixed-integer linear programming (MILP) is being applied. This restricts each element of optimisable vector $\mathbf{x}$ to only take integer values, which is suitable in this instance as one may define any unit for power consumption to a suitable resolution and the processes may be viewed as a set of binary states. By applying this restriction, a more extensive number of constraints may be applied.

3. Program

The program is separated into two routines. Requirements are detailed for both the lifetime and a single orbit. The optimisation routine then seeks to satisfy these settings, with some requirements being set by the lifetime optimisation routine for the single orbit routine. Illustrations are given of the requirements, which will be converted to the constraint form of the MILP procedure, i.e. Eqs. (2) and (3). A state of a component at a certain time will be given by $x_{i,j}$, where i denotes the component number, and j denotes some time variable, be it an orbit number or a time step. Time is given in a discrete format.

Both routines will produce a schedule of each component's usage. The lifetime routine details which components will be implemented during each orbit, and the number of times the component will be switched on. The orbital routine details which components will be switched on for a certain time step of an orbit.

3.1. Lifetime optimisation routine

The lifetime routine analyses the energy required by the satellite subsystems to complete its operations during the mission lifetime. This is subject to the mission requirements of each individual component. An objective function may be given by,

$$f(\mathbf{x}) = E\mathbf{x}, \quad (4)$$

where $\mathbf{x}$ is an array of component states for each orbit, covering the entire lifetime, and E is an array of the corresponding energies for each component state. The energy required by a component state is the power of the item multiplied by the time it will remain on during an orbit. This energy is assigned to a position in E , where it will correspond to a certain component state.

As a first case, an experiment of MIST, CUBES, will need be in an on state a set number of times during the lifetime. So, letting $i = 1$ and $x \in \{0,1\}$, a constraint will be formed,

$$x_{1,1} + x_{1,2} + x_{1,3} + \dots + x_{1,T} = N, \quad (5)$$

where T gives the total number of orbits and N is the desired number of times the CUBES system is to be run during the mission duration. A position in the energy array E , of Eq. (4), will correspond to the energy required for the system to be on for the duration of the orbit.

Another experiment on MIST is RATEX-J. To be operated successfully during an orbit, it must be run for at least 30% of the orbital time. This experiment is not constrained by an orbit

number, but needs to be run for at least a pre-set period during the satellite lifetime. This constraint may be given by

$$0.3x_{2,1} + 0.3x_{2,2} + \dots + 0.3x_{2,T} \leq R, \quad (6)$$

where R is the minimum time required for RATEX-J to stay on and is given as a fraction of number of orbits. Due to the integer nature of MILP, $x \in \{0,1,2,3\}$ and so only may be in operation for 30%, 60% or 90% of the orbit. By providing different coefficients to those in Eq. (6), and changing the domain of x , a different set of operations may be possible for RATEX-J.

The SiC experiment holds a similar restriction to RATEX-J. It must be run for a period of 60 s every hour at least while the satellite is in orbit. So, a constraint may take the form,

$$x_{3,1} + x_{3,2} + \dots + x_{3,T} \geq 1.5T. \quad (7)$$

A single orbit takes approximately 90 min, and so SiC should be run $1.5T$ times, hence the equality. The optimisation variable should consequently be constrained to $x \in \{1,2\}$.

A more complex component present on MIST is LEGS. According to a specification, this experiment must complete 8214 cycles during the satellite's life, where each cycle runs for 10 s. So, a constraint may take the form,

$$x_{4,1} + x_{4,2} + \dots + x_{4,T} = 8214, \quad (8)$$

where $x \in \{0,1,2, \dots, l\}$ to some reasonable upper limit l . However, an additional constraint must be placed on LEGS. Due to restrictions of the power circuit, SiC must be in operation for LEGS to be used, and so an additional constraint is,

$$6x_{4,j} - x_{3,j} \geq 0, \quad (9)$$

where j is the orbit number. Each optimisable variable corresponds to the number of times that component is run during an orbit, so $x_{4,j}$ is multiplied by 6 since SiC is run for 60 s and LEGS is run for 10 s.

The essential constraint for this routine is to ensure the total energy of all the components during each orbit does not exceed the total energy available during an orbit. This constraint is summarised by,

$$E_1x_{1,j} + E_2x_{2,j} + \dots + E_nx_{n,j} \leq E_{\text{sun}}, \quad (10)$$

where n is the total number of components and E_{sun} is the available energy from an orbit, taking it to come from solar panels. This constraint must be repeated for each orbit of the satellite.

3.2. Orbital optimisation routine

The orbital routine analyses the power and energy required by each component to complete its operations during a single orbit. Similar to the lifetime routine, the objective function considers the total energy, taking the form,

$$f(\mathbf{x}) = \mathbf{P}\mathbf{x}\Delta t, \quad (11)$$

where $\mathbf{x}$ is an array of component states for each time step, covering the entire orbit, $\mathbf{P}$ is an array of the corresponding power required for that component and Δt is the size of the time step. The state of each optimisation variable in $\mathbf{x}$ is strictly restricted to $x \in \{0,1\}$, detailing if the component state

is 'ON' or 'OFF'. This routine is dependent on results of the lifetime routine, and will be referred to if needed.

As with the previous routine, consider the CUBES system, which needs to be on for the entire orbit. This is dependent on the previous routine, dependent whether it has been set to on. For the orbital routine, it must be then constrained to,

$$x_{1,1} + x_{1,2} + x_{1,3} + \dots + x_{1,T} = T, \quad (12)$$

where T is the number of time increments in an orbit.

RATEX-J, as described previously, must be on for a specified percentage of the orbit, which is at least 30%. Letting this percentage be represented by s , the condition will take the form,

$$x_{2,1} + x_{2,2} + \dots + x_{2,T} = sT, \quad (13)$$

where the equality is to the number of time increments to satisfy the orbital percentage. It must however be ensured that the time RATEX-J is on is continuous, i.e. on continuously for the duration of the orbital fraction specified. A constraint will take the form,

$$sTy_{2,j} - (x_{2,j} + x_{2,j+1} + \dots + x_{2,j+sT}) \leq 0, \quad (14)$$

where a new variable $y_{i,j}$ has been introduced. This new variable is necessary to develop the condition in Eq. (14), and essentially defines a time position where RATEX-J will switch on. As RATEX-J will only be switched on once, a very simple condition is developed,

$$y_{2,1} + y_{2,2} + \dots + y_{2,T} = 1. \quad (15)$$

This variable y , in which is given a subset $y \in \{0,1\}$, only needs to be created when a subsystem is to be used for only part of the orbit.

Different to RATEX-J, SiC may be run once or twice in each orbit, so a slight modification to Eqs. (13) - (15) is required. Letting N be the number of times SiC is to be switched on each orbit, writing,

$$x_{3,1} + x_{3,2} + \dots + x_{3,T} = \frac{60N}{\Delta t}, \quad (16)$$

creates a condition for SiC to be run for 60 s for each on that is required during the orbit. A variation of Eq. (14) is then created so that the on period is maintained for a period of 60 s, and then switched off, given by,

$$\frac{60y_{3,j}}{\Delta t} - (x_{3,j} + x_{3,j+1} + \dots + x_{3,j+\frac{60}{\Delta t}}) \leq 0. \quad (17)$$

The variable y is again used to provide a time step that SiC will be in operation. Depending on how many times SiC will be run, variables y are constrained to,

$$y_{3,1} + y_{3,2} + \dots + y_{3,T} = N. \quad (18)$$

According to the specified cycle number by the lifetime routine, LEGS will need to be run a certain number of times in an orbit. So, depending on this number n , a constraint will take the form,

$$x_{4,1} + x_{4,2} + \dots + x_{4,T} = \frac{10N}{\Delta t}, \quad (19)$$

since each cycle runs for 10 s. Again, each cycle must be run for a period of 10 s continuously, so developing a variable y ,

$$\frac{10y_{4,j}}{\Delta t} - (x_{4,j} + x_{4,j+1} + \dots + x_{4,j+\frac{10}{\Delta t}}) \leq 0. \quad (20)$$

Variable y is providing a time step that LEGS will begin a cycle on, where another constraint is required of the form,

$$y_{4,1} + y_{4,2} + \dots + y_{4,T} = N. \quad (21)$$

As discussed before, LEGS may only be operated whilst SiC is on. An additional constraint is required to achieve this, which requires that the state of SiC is on whenever LEGS is run,

$$x_{3,j} - x_{4,j} \geq 0. \quad (22)$$

A primary constraint for the operation of the satellite is the restriction of the battery. When the satellite is in sunlight, the battery is charging. The number of time steps the satellite is in eclipse is denoted by e . To simplify the constraint, a new orbit begins when the satellite leaves the eclipse. So, the energy the battery will store in an orbit is found by,

$$P_1 x_{1,1} \Delta t + P_2 x_{2,1} \Delta t + \dots + P_1 x_{1,2} \Delta t + \dots + P_n x_{n,T-e} \Delta t + B = E_{\text{sun}} \quad (23)$$

where B is the energy stored in the battery and E_{sun} is the energy taken in from the solar panels. During the eclipse, the only power source is the battery, and so during the eclipse period, the components will be constrained to the energy not exceeding that available from the battery,

$$P_1 x_{1,T-e} \Delta t + P_2 x_{2,T-e} \Delta t + \dots + P_1 x_{1,T-e+1} \Delta t + \dots + P_n x_{n,T} \Delta t - B \leq 0. \quad (24)$$

Maintaining all these constraints will ensure successful operation during the orbit that will not exceed available power. These constraints may be easily manipulated to ensure other requirements are satisfied.

4. Implementation and Testing

The scenario used for testing the performance of the algorithm was the MIST satellite in a Low Earth Orbit (LEO), with an orbital period of approximately 98 minutes. The total optimisation scope was set to one year, allowing for 5000 orbits. The input energy from the solar panels was assumed for this orbit to average 11.5 Wh over the time whilst not in eclipse. During eclipse, the energy from the sun was set to zero as it was regarded as negligible. The capacity of the battery-pack assumed in the simulation was 6.9 kWh. The MIST-satellite may use twice this capacity, but this value was set as a worst case.

Following the lifetime optimisation routine, the results of the $\mathbf{x}$ optimisation array were placed in a two-dimensional matrix $\mathbf{LS}$. These results were then sent as input to a function containing the orbit optimisation routine. For each orbit, this function would apply the run requirements imported from $\mathbf{LS}$.

The orbit optimisation function would then allocate the requested processes and return the full schedule for the orbit. The time discretisation was set to 10 s, resulting in 588 time increments per orbit cycle.

The function also calculated the depth-of-discharge for the proposed battery module. These discharge-levels may then be

used as a means of verifying the algorithms performance. The battery's performance decline over time was neglected in this analysis.

The final value for the depth-of-discharge was returned to the main function to be used as the first discharge in the following orbit. The change in charge caused in each time increment was calculated by taking the dot product of the vector containing the run-schedule for the specific time-increment j and the one containing the power requirements for the components and multiplying them by the length of the increment,

$$\Delta B_j = (\mathbf{S}_j \cdot \mathbf{P}) \Delta t. \quad (25)$$

The power level in each increment may then be found by,

$$B_j = B_{j-1} + \Delta B. \quad (26)$$

The resulting schedule for the entire lifetime of the satellite showed an even distribution of power over the orbits as seen in Fig. 1 showing the first ten orbits.

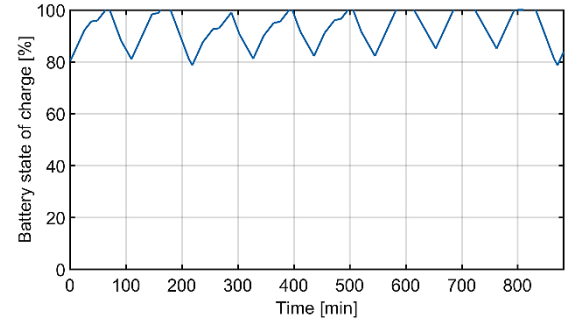


Fig. 1. The first ten orbits battery state of charge estimation. Time along the x-axis is presented in minutes.

The full table, shown in Fig. 2 shows that no major spikes nor lows were present over the course of the entire simulated year.

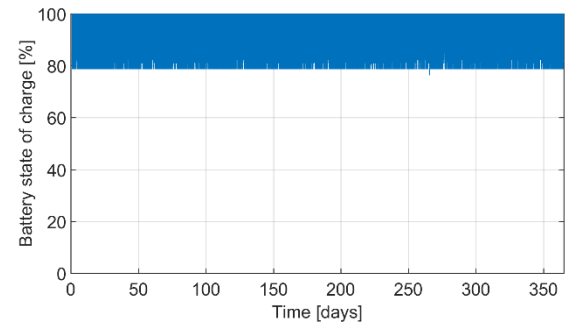


Fig. 2. The full lifetime battery state of charge estimation. Time along the x-axis is presented days. Individual orbits not distinguishable in cluster, but major deviations are discernible.

The performance was examined by looking at the depth of discharge achieved at the exit of each eclipse which was consistently when battery charge would be at its lowest during an orbit.

The battery's lowest level of charge was 76.35% Average

energy output was 6.25 Wh and the average input was 7.04 Wh showing that 88.75% of all input energy being used for running the proposed schedule.

A total of 8 experiments plus subsystems, transmitter array and partial processes adding up to a total of 15 optimisable components were scheduled over 2940000 time-increments. When run on a stationary computer using an Intel i4 processor the runtime for the program was timed as ~3.2 hrs. Average time for individual orbit optimisation routines was 2.3 seconds and the parent function containing the lifetime optimisation routine took 73.5 seconds.

5. Conclusion

This paper has presented the method to utilize mixed integer linear programming to derive a suitable process-schedule, for a set amount of time on a satellite or closed system, with limited power input. Some simplifications have been made with regards to the performance of a satellite's subsystem. Under the conditions imposed on the algorithm it appears to handle the scheduling without much difficulty and shows promise for good performance when faced with more accurate test models as well.

The method has been examined in a steady state with one fixed schedule being generated by predetermined conditions. In the current state one could utilize the program on Earth to generate a schedule which can be converted and uploaded to the satellite from a ground station.

However, potential for further development may be seen in the structure surrounding the algorithm, which could allow for it to become real time and adaptive on-board the satellite. This would make the system more self-reliant, leading to more effective use of the time in orbit.

It is suggested that this may be done by allowing for sensor inputs into some monitoring program. Such a program would adjust the program's constraint values in real-time. For instance, a sensor reading may be from monitoring the charge generated from the solar panels during each orbit. The suggested approach would be particularly useful during longer missions,

as relevant subsystems, such as batteries, tend to change in performance during the satellite lifetime.

One scenario may be to consider the declining performance of the battery module, as the peak charge decreases and so will the safe range of charge to operate within. It would be desirable to plan the processes in such a way that it will attempt to maximize the remaining performance level of the module. If necessary the satellite could do an on-board rescheduling of its remaining lifetime and generate the schedule for all increments of the following orbit, limiting the computational strain for any individual orbit to at most two instances of optimisation.

Another suggestion is to adapt some requirements to be based on geographical location through a GPS positioning measurement. This will allow for a more responsive system. For example, when the satellite is within range of a ground station, the transmitter should be set to switch on. Using just a time position is less direct.

Even though scope for improvement is suggested, the success of the MILP approach is illustrated. The algorithm allows for efficient and effective review of scheduling for satellite power systems.

Acknowledgements

We wish to thank the KTH Space Centre, and, in particular, the contributions of Sven Grahn, Christer Fuglesang and the MIST-team for their input, advice and general support throughout the production of this paper.

References

- 1) Xhafa, F., Sun, J., Barolli, A. Biberaj, A., and Barolli, L.: Genetic Algorithms for Satellite Scheduling Problems, *Mobile Information Systems*, **8** (2012), pp. 351-377.
- 2) Pemberton, J. C. and Galiber, F.: A Constraint-Based Approach to Satellite Scheduling, *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, **57** (2001), pp. 101-114.
- 3) Arnold, S. S., Nuzzaci, R., and Gordon-Ross, A.: Energy Budgeting for CubeSats with an integrated FPGA, *Proceedings of the IEEE Aerospace Conference, Big Sky*, 2012, pp.1-14.